

Python

→ we make a folder python name.

↳ then 01-BASIC-PYTHON.

↳ hello-sharawal.py
↳ sharawal.py.

hello-sharawal.py:-

```
print("rahul sharawal");
```

```
def rahul(n):  
    print(n)
```

```
rahul(4)
```

→ sharawal.py:-

```
from hello-sharawal import rahul;
```

```
rahul("python start");
```

→ study about this.

→ It make new file with name --pyche-- (new folder)
inside it new file:-

hello-sharawal.(python-314.py)

Python's inner working:-

→ Python sharwal.py :- we can call program / script.



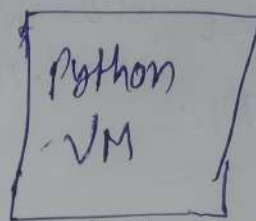
⇒ Byte Code



mostly hidden

(ex:- --pycache-- it mostly hidden but some cases it show like import)

⇒



Virtual Machine

IE actually run code.

① Compile to Byte Code :- (It not mean it compile language)
(it's only a tech term)

Byte code is low level & Platform Independent.

→ Byte code runs faster.

• pyc --- → compiled python (Frozen Binaries)

--pycache-- (according to change in code .pyc delete & make version so --pycache-- use for arrange this all files.)

source change (.pyc use difing algorithm) means it not change all code it change new things like github.

hello.sharwal.py (python -bm.pyc)

↓
source change

↘
version of python based on C.

→ Works only for imported files.

→ Not for top level files (means only one file all code)

Python Virtual Machine (PVM) :- We call Python interpreter language means line by line run.
→ Code loop to ~~inter~~ iterate byte code (Continuously work & when we give any code it execute)
→ Run time engine
→ Also known as Python interpreter.

Byte code is not Machine code.

- Python specific interpretation
- CPython, Jython, Iron Python, Stackless, Pypy
- standard implementation.

Python in shell :-

IDLE (Integrated Development & Learning Environment) :-
→ is Python's built-in editor & interpreter used for writing, running & testing Python programs.

→ Terminal :- Here we test, ~~not~~ not for code.
Like thing $2+4$ so want to check o/p).

→ Python
→ `print("rahul")`.

→ In Python we can import multiple modules.
→ other written code. we can import that called modules.

⇒ :- (as all things come here).

>>> `import os` (as all things come here).

>>> `os.getcwd()` (~~as~~ get current working directory).

~~/Users/~~ 'C:\Users\rahul\Documents\Python\Python\01-basic Python'

>>> `for c in "chai":`

... `print(c)`

Indentation ~~error~~ → means we need to click tab.

>>> `for c in "chai":`

... `print(c)` (after ... click tab than enter).

... (enter)

c
h
a
i

System module:-

```
>>> import sys  
>>> sys.platform (→ Here we not use () because it variable)  
'win32'
```

```
>>> import hello, shawwal  
shahul shawwal  
4  
>>> hello & shawwal. shahul ("mint tea")  
mint tea
```

hello, shawwal.py:-

```
chai-one = "lemon tea"  
chai-two = "ginger tea"  
chai-three = "masala chai".
```

```
>>> hello, shawwal.chai-one
```

AttributeError because it not load that/import that ~~is~~ new code
means when we import this ~~at~~ all things not present there.
So we need to import again or load this all things.

```
>>> from importlib import reload
```

```
>>> reload(hello, shawwal)
```

python max. data type is object (Immutable).
 Mutable means changeable. & Immutable means not change.

Mutable & Immutable data types in python

Mutable DT

- * List
- * set
- * Dictionary
- * Bytearray
- * Array

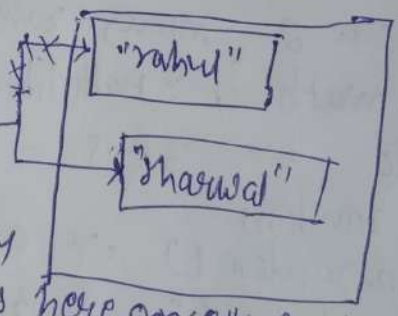
Immutable DT

- * Integers
- * Floating-point numbers
- * Boolean
- * string
- * Tuples
- * Frozen set
- * Bytes.

Confusion :- Python (in terminal)

```
>>> username = "rahul"
>>> username
'rahul'
>>> username = "sharwal"
>>> username
'sharwal'
```

Integers or string is Immutable but here it is change ?



Immutable means once a object assign space in M/M then that not rewrite means this we assign sharwal it's not

Why not here y change ? here once "rahul" assign after mean we overwrite on rahul it make new object, "sharwal".

```
>>> x = 10
>>> y = x
>>> x
10
>>> y
10
>>> x = 15
>>> y
10
```


Data Types / Object Type:-

5

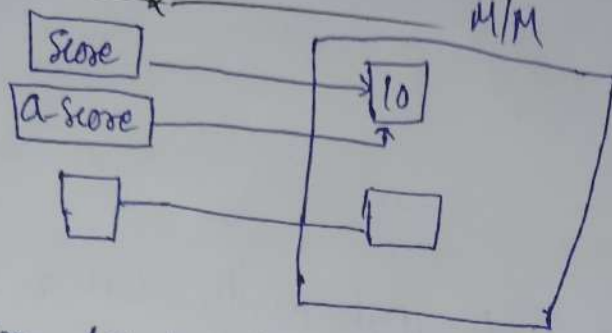
- * Number :- 1234, 3.1415, 3+4j, 0b111, Decimal(), fraction()
- * String :- 'spam', "Bob's", b'a\x01c', u'sp\x04m'
- * List :- [1, 2, 'three'], 4.5, list(range(10))
- * Tuple :- (1, 'spam', 4, 'V'), tuple('spam'), namedtuple
- * Dictionary :- d'food': 'spam', 'taste': 'yum', dict(howegs=10)
- * Set :- set('abc'), d'a', 'b', 'c'
- * File :- open('eggs.txt'), open(r'C:\ham.bin', 'wb')
- * Boolean :- True, false
- * None :- None

* Functions, modules, classes

* Advance :- Decorators, Generators, Iterators, Metaprogramming.

```
>>> 2 ** 100 means 2100
>>> 'rahul' * 2 means rahul two times => 'rahulrahul'
>>> import math <-> math library import now we can use pi etc
>>> math.pi => 3.1415...
>>> import random
>>> random.random() => 0.anything
>>> random.choice([1, 2, 3, 4, 5]) <-> It give random number from list
>>> username = 'rahul'
>>> len(username) <-> give length 5
>>> username[0] = 'A' -> give error b/c it is immutable.
>>> username[-1] <-> It give last number of value here l
>>> username[1:3] <-> It give 1 to 3 number here:- ah
>>> dir(username) => It give what we can use
>>> mylist = [123, 'rahul', 3.14] <-> List
>>> myD = d'one': 'rahul', 'two': 'sharwal', 'fullname': 'Rahul Sharwal' <-> Dictionary
>>> myTup = (1, 2, 4)
>>> myTup[0] => 1
```


Internal Working of Python:- When we assign a value that store in M/M but when we delete it than only we delete index/assign name. Value is delete using Python Garbage that remove that value that not have ref.



→ We can count a ref using `sys.getrefcount()`. ex:- first import `sys` then `sys.getrefcount(a)`
 ref/index not have datatype only m/m have datatype
 ex:- here `score` not have any datatype but `10` have datatype number.
 means Python not have DT only m/m have DT

Python Garbage: Numbers & strings not delete immediately b/c it thing if need again assign that.

ex:- `a=3`, `a=10` means `3` not delete immediately it delete some time later

```
>>> l1 = [1, 2, 3]
>>> l2 = l1
>>> l1 => o/p :- [1, 2, 3] & l2 => o/p [1, 2, 3]
>>> l1[0] = 55 now l1 = [55, 2, 3] also l2 = [55, 2, 3]
    because both ref is same.
>>> m1 = [1, 2, 3]
>>> m2 = m1
>>> m2 = [1, 2, 3] => now ref is change.
>>> m1[0] = 44 now m1 = [44, 2, 3] but m2 = [1, 2, 3] because ref-
    - is change.
>>> m3 = m2[:] it means m3 take copy of m2 & ref is diff.
>>> import copy
>>> m4 = copy.copy(m2) { -> same m3 = m2[:] }
>>> m = [1, 2, 3]
>>> n = m
>>> m == n true (o/p)
>>> m is n true (o/p)
>>> n = [1, 2, 3] now m == n true but m is n not true.
```

Diff b/w Tuple & List :-

List = [] Tuple = ()

⇓

⇓

Mutable

Immutable

Why we need tuple, because we have List that easily change but when we need immutable then we use tuple that give us reference from m/m that does not change new reference are making.

DT

Syntax

Integer → x = 1234

float → x = 3.1415

complex → x = 3+4j

Binary → x = 0b111

Decimal → x = Decimal('3.14')

Fraction → x = Fraction(3,4)

String → x = 'spam'

Bytes → x = b'a\x01c'

Tuples → x = (1, 'spam', 4, 'U')

List → x = [1, [2, 'three'], 4.5]

Dictionary → x = {'food': 'spam', 'taste': 'yum'}

Set → x = {'a', 'b', 'c'}

File → x = open('eggs.txt')

Boolean → x = True

None → x = None